Agent Phantom Recovery

07-trd.md

Technical Requirements Document (TRD)

Version: 1.0
Status: Draft
Document Type: Production Technical Requirements Document

---

1. Purpose

1.1 Objective

This document defines the technical requirements for Agent Phantom Recovery.

Agent Phantom Recovery is a long-horizon autonomous engineering and security recovery platform designed to investigate, reason, execute, verify, and complete complex technical tasks through the coordinated use of large language models, tools, memory systems, and execution environments.

Unlike conventional AI assistants, the system is designed around task completion rather than conversational interaction.

The platform must be capable of:

- Understanding technical goals
- Planning multi-step workflows
- Using tools repeatedly
- Adapting to observations
- Maintaining long-term task state
- Investigating repositories
- Diagnosing failures
- Designing fixes
- Implementing patches
- Running verification procedures
- Producing engineering deliverables

The primary success criterion is completion of engineering objectives rather than generation of natural language responses.

---

2. Scope

2.1 Included Capabilities

The platform shall support:

Engineering Tasks

- Repository analysis
- Bug investigation
- Root cause analysis
- Code modification
- Testing
- Validation
- Refactoring support
- Architecture understanding

Security Tasks

Authorized defensive security activities including:

- Vulnerability discovery
- Security code review
- Misconfiguration detection
- Risk assessment
- Remediation generation
- Verification of fixes

Agentic Tasks

- Long-running workflows
- Multi-step execution
- Tool orchestration
- Memory-driven continuation
- Dynamic replanning

2.2 Out of Scope

The system is not intended to be:

- A general-purpose chatbot
- A social assistant
- A content-generation platform
- A one-shot code scanner
- A static report generator

The system must prioritize execution and verification over conversation.

---

3. Product Objectives

3.1 Primary Objective

Enable autonomous completion of engineering tasks through:

```
Goal
 ↓
Plan
 ↓
Tool Use
 ↓
Observe
 ↓
Replan
 ↓
Verify
 ↓
Deliver
```

3.2 Secondary Objectives

Objective A

Reduce engineering investigation time.

Objective B

Increase bug-fix accuracy.

Objective C

Improve vulnerability remediation quality.

Objective D

Maintain context during long-running tasks.

Objective E

Generate production-ready deliverables.

---

4. Technical Principles

4.1 Tool-First Architecture

The system shall prefer deterministic evidence collection before model reasoning.

Examples:

- Repository indexing
- Static analysis
- Test execution
- Log collection
- Dependency analysis

Reasoning should occur only after evidence is gathered.

4.2 Verification-First Design

No major finding or patch should bypass verification.

Every significant action must pass validation checks.

4.3 Long-Horizon Operation

The architecture shall support:

- Extended execution sessions
- Task continuation
- Context persistence
- Multi-stage workflows

4.4 Explainable Execution

The platform shall expose:

- Current task state
- Active plan
- Tool activity
- Evidence sources
- Verification results

Users must be able to understand why actions are taken.

4.5 Modularity

Major system components shall remain independently replaceable.

Examples:

- LLM provider
- Memory provider
- Search provider
- Tool provider
- Execution environment

---

5. Functional Requirements

FR-1 Task Intake

Description

The system shall accept a technical objective from the user.

Inputs

- Text instruction
- Repository
- File set
- URL
- Existing session

Outputs

- Structured task object

FR-2 Goal Interpretation

The system shall convert user requests into internal execution goals.

Example:

User:

```
Fix failing authentication tests
```

Internal Goal:

```
Investigate authentication subsystem
↓
Run tests
↓
Identify failure
↓
Generate patch
↓
Verify fix
```

FR-3 Planning

The system shall generate executable plans.

Plans must contain:

- Steps
- Dependencies
- Tool requirements
- Success conditions

Plans must be updateable.

FR-4 Tool Selection

The system shall determine which tools are required.

Supported tool categories:

- Terminal
- Browser
- Search
- File System
- Code Execution
- Git
- Repository Analysis

Tool selection shall be evidence-driven.

FR-5 Repository Analysis

The system shall support repository understanding.

Capabilities include:

- Structure analysis
- Dependency mapping
- Call graph generation
- File ranking
- Context extraction

FR-6 Source Code Understanding

The system shall analyze:

- Python
- Go
- Rust
- Java
- C++
- JavaScript
- TypeScript

Future language support must be extensible.

FR-7 Browser Interaction

The system shall support integrated browser operation.

Capabilities:

- Open pages
- Navigate pages
- Inspect content
- Observe UI behavior
- Capture evidence

Browser state shall remain visible within the workspace.

FR-8 Terminal Interaction

The system shall support integrated terminal execution.

Capabilities:

- Run commands
- Execute builds
- Run tests

- Capture logs
- Install dependencies

Terminal history shall remain accessible.

FR-9 File System Interaction

The system shall:

- Read files
- Create files
- Modify files
- Delete files
- Search files

Actions must be logged.

FR-10 Search Capability

The platform shall support retrieval from:

- Documentation
- Project artifacts
- Internal indexes
- Approved web resources

Search results shall be attributable.

FR-11 Patch Generation

The system shall be able to:

- Generate code fixes
- Modify existing code
- Create patches
- Explain modifications

Every patch must contain rationale.

FR-12 Test Execution

The platform shall support:

- Unit tests
- Integration tests
- Build verification
- Lint verification

Results must be stored as evidence.

FR-13 Root Cause Analysis

The system shall identify:

- Immediate cause
- Contributing factors
- System impact
- Recovery strategy

Root cause analysis shall be linked to evidence.

FR-14 Verification

Verification shall include:

- Test execution
- Evidence review
- Patch review
- Regression checks

Verification must occur before completion.

FR-15 Final Deliverables

The system shall generate:

- Reports
- Patches
- Pull request summaries
- Investigation results
- Verification records

Deliverables must be reproducible.

---

6. Agent Requirements

AR-1 Autonomous Operation

The system shall execute tasks without requiring user input after every step.

AR-2 Adaptive Replanning

The system shall modify plans when new evidence appears.

Example:

```
Plan A
 ↓
New Evidence
 ↓
Plan A Rejected
 ↓
Plan B Generated
```

AR-3 Multi-Step Persistence

The system shall support:

- 20+ execution steps
- Long workflows
- State continuity

without losing task objectives.

AR-4 Observation Loop

The platform shall operate through:

```
Observe
↓
Act
↓
Observe
↓
```

```
Act
↓
Observe
```

rather than single-pass execution.

AR-5 Goal Retention

The original objective shall remain available throughout execution.

Goal drift must be minimized.

---

7. Multi-LLM Requirements

MLR-1 Planner Model

Responsibilities:

- Goal decomposition
- Tool selection
- Workflow management
- Progress tracking

MLR-2 Reasoner Model

Responsibilities:

- Deep analysis
- Root cause identification
- Patch generation
- Architecture reasoning

MLR-3 Verifier Model

Responsibilities:

- Challenge findings
- Review fixes
- Detect regressions
- Approve or reject outcomes

MLR-4 Independent Verification

Verification shall be performed by a model distinct from the primary reasoning model.

This reduces correlated failures.

---

8. Technical Acceptance Criteria

The system shall be considered technically successful when it can:

Capability 1

Accept a repository and investigate it.

Capability 2

Generate an execution plan.

Capability 3

Use tools repeatedly.

Capability 4

Maintain context during long tasks.

Capability 5

Produce a code modification.

Capability 6

Verify the modification.

Capability 7

Generate a final engineering artifact.

Capability 8

Recover from failures and continue toward completion.

End of TRD Part 1

Next Section:

- Memory Requirements
- Reasoning Requirements
- Verification Requirements
- Security Requirements
- Browser Requirements
- Terminal Requirements
- Workspace Requirements
- Infrastructure Requirements
- Reliability Requirements
- Scalability Requirements